

TALENT INTELLIGENCE PLATFORM

Distributed Microservices Architecture & System Design Interview Study Guide

A deep architectural dissection of your production-grade backend platform.

This guide maps core system design topologies, inter-service API Gateway routing, security context propagation, Redis caching strategies, Kafka idempotent consumer boundaries, and RAG-enabled chat pipelines directly to your Spring Boot 3.4.0 and Java 21 implementation.

Prepared for senior engineering interviews, system audits, and architectural analysis.

CONFIDENTIAL SYSTEM ARCHITECTURE HANDBOOK
DESIGNED FOR HIRING PREPARATION & ARCHITECTURAL AUDIT

SECTION 1 - SYSTEM TOPOLOGY & MICROSERVICES GRID

Microservices Grid Overview

The platform implements a highly decoupled, database-per-service distributed topology built on Java 21, Spring Boot 3.4.0, and Spring Cloud modules. Service discovery is managed by Eureka Server on port 8761, exposing endpoints through a Netty-based Spring Cloud Gateway running on port 8090.

Each downstream service encapsulates its domain boundaries entirely. Direct database sharing is prohibited, ensuring schema modifications on one service (e.g. adding columns to resume table) do not trigger cascading breaks. Communication alternates between synchronous REST (Feign) for low-latency CRUD metadata and asynchronous Kafka events for high-throughput heavy calculations.

Microservice Directory Mapping

- * eureka-server (8761): Service discovery lookup registry.
- * api-gateway (8090): Spring Cloud Gateway for JWT auth verification, routing, and header claims injection.
- * user-management-service (8081): Recruiter profiles, DB = user_db.
- * authentication-service (8082): Password validation, token signing (JwtS), OTP generation, DB = auth_db.
- * resume-management-service (8083): File uploads (AWS S3), CandidateResume state controller, DB = resume_db.
- * ai-screening-service (8084): Apache Tika parsing, sentence chunking, Vector embeddings (PgVector), DB = screening_db.
- * candidate-ranking-service (8086): Ollama-based scoring, Redis Sorted Set rankings, DB = ranking_db.
- * recruiter-chat-service (8087): RAG pipelines (expansion, hybrid search, cosine rerank), DB = chat_db.
- * notification-service (8088): Kafka email consumers (no database).

SECTION 2 - CORE ARCHITECTURAL PATTERNS

A. API Gateway Authorization Context Propagation

Rather than validating JWT tokens inside every single service, the Netty-based api-gateway intercepts requests, validates the signature using HMAC-SHA key parsing, extracts user profile parameters, and translates them into standard downstream headers:

- X-User-Email
- X-User-Role
- X-User-Id

Downstream services load `GatewayHeaderAuthFilter` in their Spring Security configuration. If X-User-Email is detected, they populate `SecurityContextHolder` with `UsernamePasswordAuthenticationToken`, trusting the gateway context natively. This keeps services completely stateless and avoids redundant token parsing overhead.

B. High-Speed Cache via Redis Sorted Sets (ZSets)

To display candidate rankings instantly on the recruiter dashboard, candidate-ranking-service bypasses PostgreSQL query-time sorting. Instead, when a candidate score is calculated, the score is added to a Redis ZSet:

```
`redisTemplate.opsForZSet().add("jd:ranking:" + jobId, resumeId, matchScore);`
```

This indexes scores under the job ID. Paginating candidates requires `ZREVRANGE` commands, providing $O(\log(N) + M)$ lookup complexity. This ensures sub-millisecond page loads under high user concurrency.

C. Kafka Failover Resiliency & Consumer Idempotency

Network fluctuations can crash inter-service Feign calls or duplicate Kafka messages. The platform mitigates this through:

1. REST-to-Event Failover: In `AIScreeningService`, updating the resume status is attempted via Feign. If Feign fails, it catches the exception and publishes a `resume-status-updated` event to Kafka. The Resume service consumes this asynchronously, ensuring eventual consistency of database states.
 2. Idempotent Consumer Design: Consumers execute read-before-write checks (e.g. `existsByEmail`) backed by unique database indices. Message routing keys (like `resumeId` or `email`) guarantee that related records are processed sequentially by the same partition thread, preventing concurrency races.
-

SECTION 3 - PIPELINES & FLOW MECHANICS

A. End-to-End Ingestion Pipeline

1. Client POSTs resume to /api/v1/resumes/upload via Gateway.
2. Gateway validates JWT, propagates claims, and forwards binary to resume-management-service.
3. Resume service uploads raw file to S3, writes CandidateResume entity as 'UPLOADED', and publishes a resume-uploaded event to Kafka.
4. AI Screening service consumes event, downloads S3 stream, parses text via Apache Tika, and chunks it via sentence boundaries.
5. AI Screening calls Ollama Llama3 to extract metadata JSON, stores text chunks + PgVector embeddings, updates Resume Service to 'PARSED', and publishes a resume-parsed event.
6. Candidate Ranking service consumes 'resume-parsed', calls Ollama screening endpoint, writes ScreeningReport, indexes score in Redis ZSet, and publishes 'SCREENED' status back to Kafka.

B. RAG & Chat Retrieval Pipeline

1. Client query and resumeld reach recruiter-chat-service.
2. QueryRouterService evaluates keyword lists to choose route: CONVERSATIONAL, TOOL_CALL, or VECTOR_SEARCH.
3. If VECTOR_SEARCH, LLM rewrites the query for search optimization, then expands it into 3 query variations.
4. HybridRetrievalService executes PgVector searches. If vector matches are < 2 , it executes Feign fallback to search local keywords inside the screening DB chunks.
5. DocumentJoinerService merges results and runs Reciprocal Rank Fusion (RRF) to rank chunks based on query overlaps.
6. RerankingService computes exact cosine similarity coordinates between query and top chunks, selecting the top 5.
7. The selected context, combined with RedisChatMemory history, is injected into the prompt and sent to Ollama for the final response.

SECTION 4 - ARCHITECTURAL DECISIONS & TRADE-OFFS

Monolith vs. Microservices

- * Approach: Decoupled services (Gateway + Eureka + 8 downstream services).
 - * Trade-off: Monolith is far simpler to deploy and maintains strong transactional consistency (ACID). However, processing resumes (Tika parsing) and LLM screening are highly CPU and RAM bound. In a monolith, spikes in resume uploads would exhaust thread pools and lock recruiter login pages. Microservices allow scaling ai-screening-service independently while auth and job lookup remain highly available.
-

PgVector vs. Dedicated Vector Database (Pinecone / Milvus)

- * Approach: Storing chunk embeddings in PostgreSQL using the pgvector extension.
 - * Trade-off: Dedicated vector stores process indices slightly faster at extreme scale (billions of vectors). However, they introduce data synchronization lag, lack transactions, and require managing a separate system. PgVector allows storing candidate text chunks, relational indices, and 768-dimensional coordinates in the same database table, enabling ACID transactions and fast relational joins in a single SQL statement.
-

Kafka vs. RabbitMQ

- * Approach: Event streaming via Apache Kafka.
 - * Trade-off: RabbitMQ is easier to configure and has lower latency. However, RabbitMQ deletes messages upon consumer acknowledgment. Kafka operates as a partitioned commit log, storing events permanently. If the screening logic or embedding model changes in the future, we can reset offsets and replay historical resume upload streams to re-index the entire candidate pool without asking recruiters to re-upload files.
-

SECTION 5 - OBSERVABILITY & SYSTEM MONITORING

Pull-Based Metrics Stack

The platform integrates a complete, production-grade observability stack built on Prometheus, Grafana, and Spring Boot Actuator with Micrometer:

1. Instrumentation: Downstream services run Actuator, exposing metrics at `/actuator/prometheus` (HikariCP pool usage, JVM GC activity, CPU consumption).
 2. Exporters: Separate exporters monitor PostgreSQL databases (postgres-exporter), Redis caching instances (redis-exporter), and Kafka brokers (kafka-exporter).
 3. Aggregation & Dashboards: Prometheus scrapes these metrics at 15-second intervals. Grafana provisions visual boards for real-time JVM thread statuses, database connection pool saturation, and Kafka consumer lag.
 4. Custom Metrics: Inside RankingService, we track custom Prometheus counters (e.g. `ranking.redis.updates.total`) to audit cache hits/failures.
-